TENSOR NETWORKS PARA SIMULAÇÃO DE CIRCUITOS QUÂNTICOS

Alexandre Silva

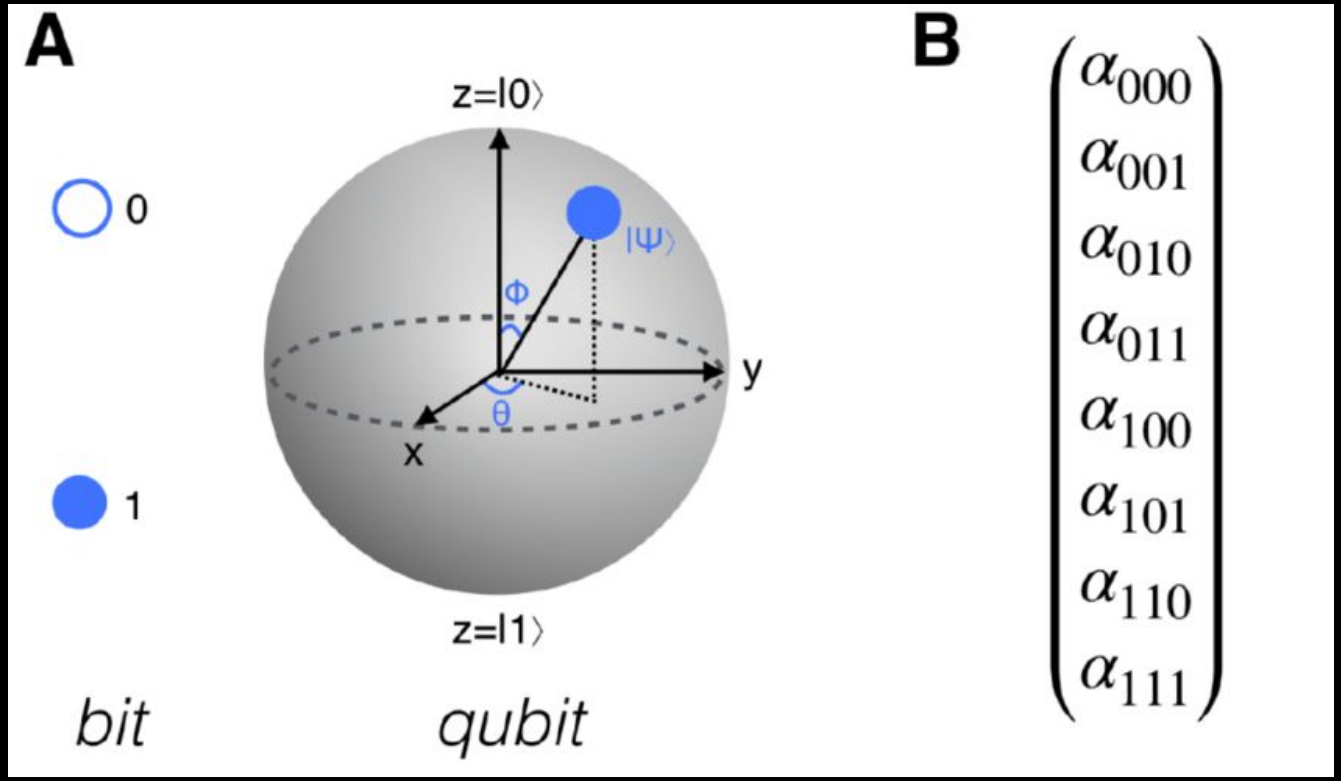
Roteiro:

1. Contextualização sobre computação quântica;
2. Entendendo o problema;
3. Entendendo Tensor Networks;
4. Algoritmo em si + Análises.

O que é a Computação Quântica?

- Utilizar a Mecânica Quântica para realizar computações;
- Âtomos, partículas, fótons, elétrons, etc;
- Possibilidade de avanços;
- Simulações mais fiéis da realidade (sistemas quânticos);
- Atualmente Computação Quântica está em evidência.

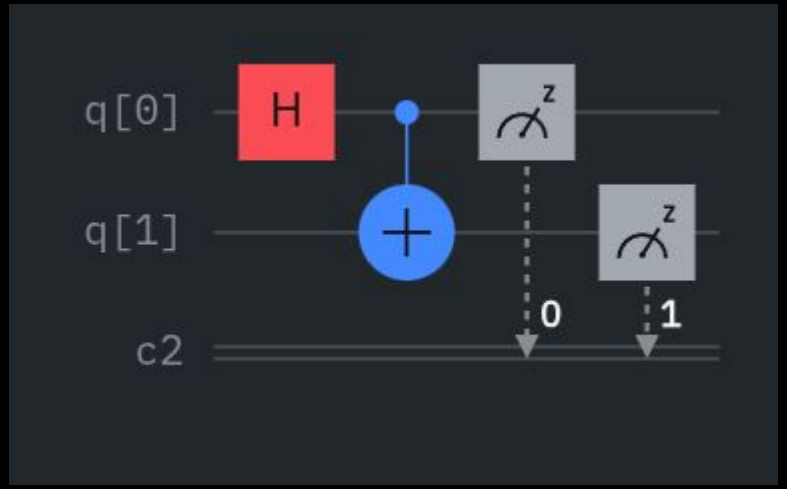
Qubits, o que são?



Fonte: XB Software

Modelo de Computação Quântica:

- Modelo de Circuito é o mais usado;
- Portas são matrizes (operadores);
- Linhas são qubits (vetores).



Fonte: Autoria própria via IBM Quantum Composer

Problema:

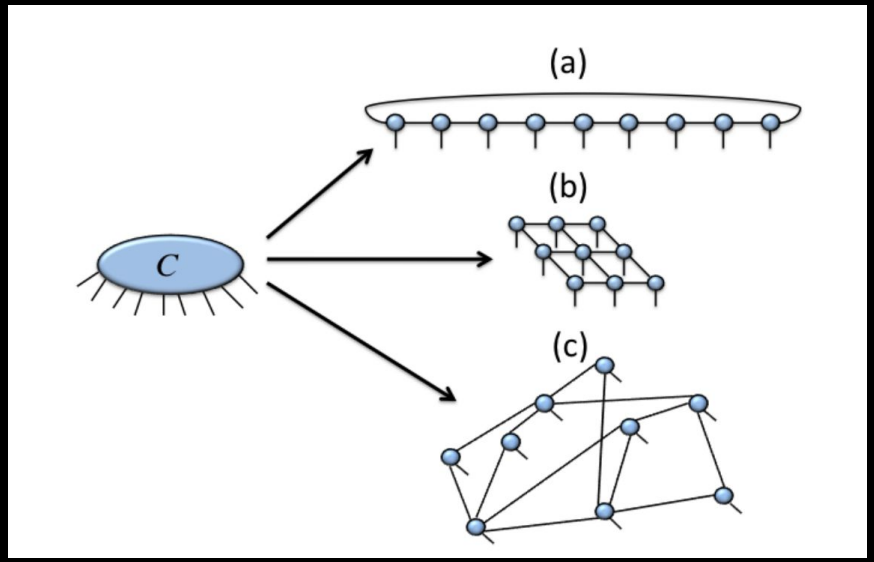
- Computadores Quânticos ainda são limitados - Noisy Intermediate-Scale Quantum (NISQ);
- Simulações para validar os algoritmos (HPC);
- Simulações são custosas
  - Possibilidade de estados cresce a $N = 2^n$
  - 40 qubits → $2^{40} \times 2^3$ bytes (float) = 2^{43} bytes = 8Tb

$$|\Psi\rangle = \sum_{i_1 i_2 \ldots i_N} C_{i_1 i_2 \ldots i_N} |i_1\rangle \otimes |i_2\rangle \otimes \cdots \otimes |i_N\rangle$$

Fonte: arXiv:1306.2164

Tensor Networks:

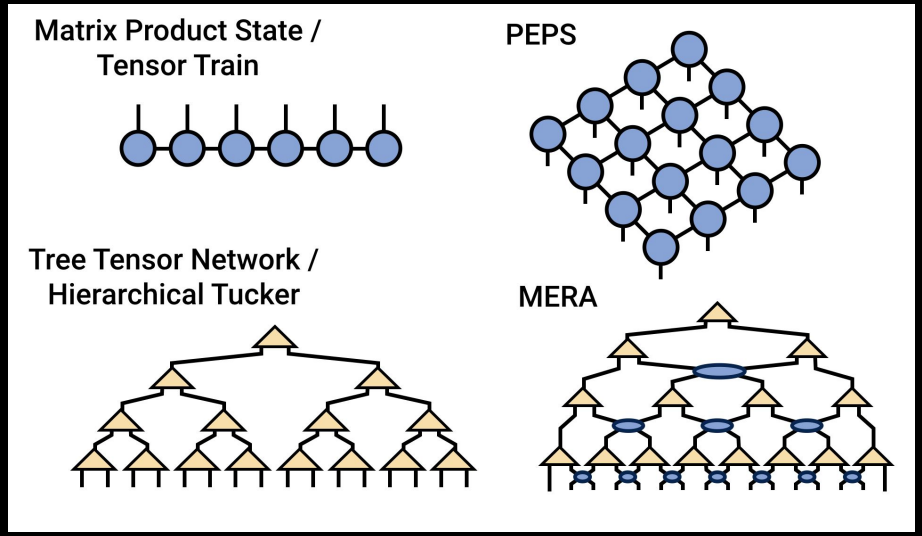
- Generalização de escalares, vetores, matrizes, etc (rank);
- Circuitos quânticos são um tipo de Tensor Network;
- Aplicar operações conforme necessário;
- Paralelizáveis.



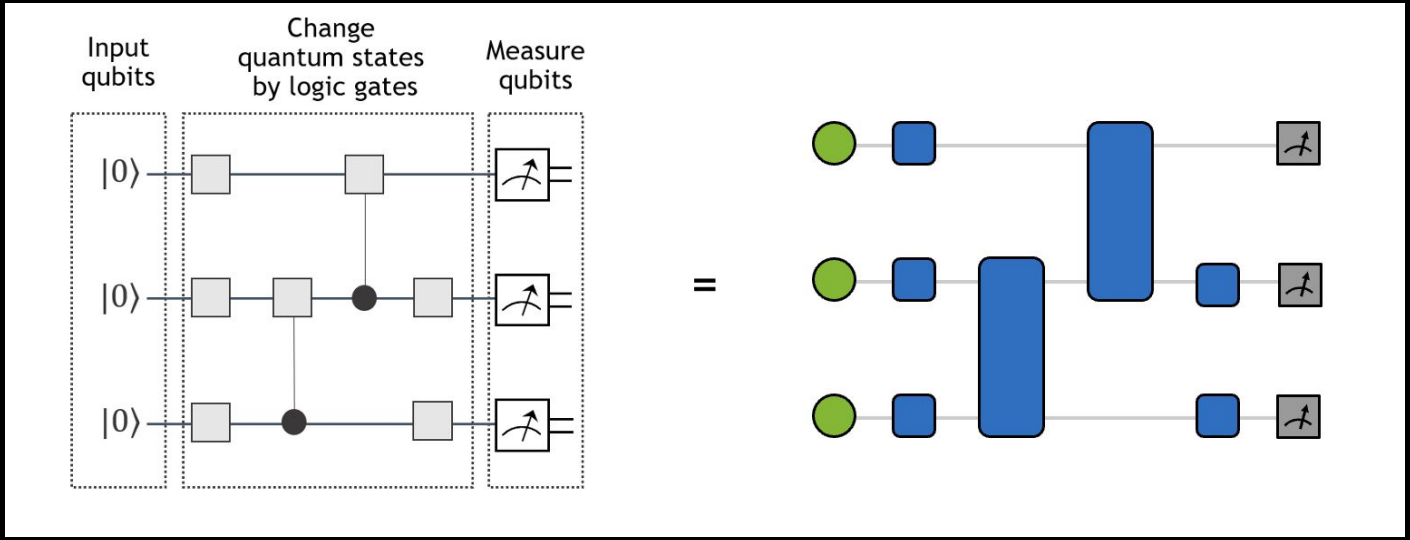
Fonte: arXiv:1306.2164

Tipos

- MPS (Tensor Train);
  - MPO;
  - PEPS;
  - PEPO;
- TTN;
  - MERA;
- Outros formatos estabelecidos ... ;
- De formato arbitrário.



Fonte: tensornetwork.org



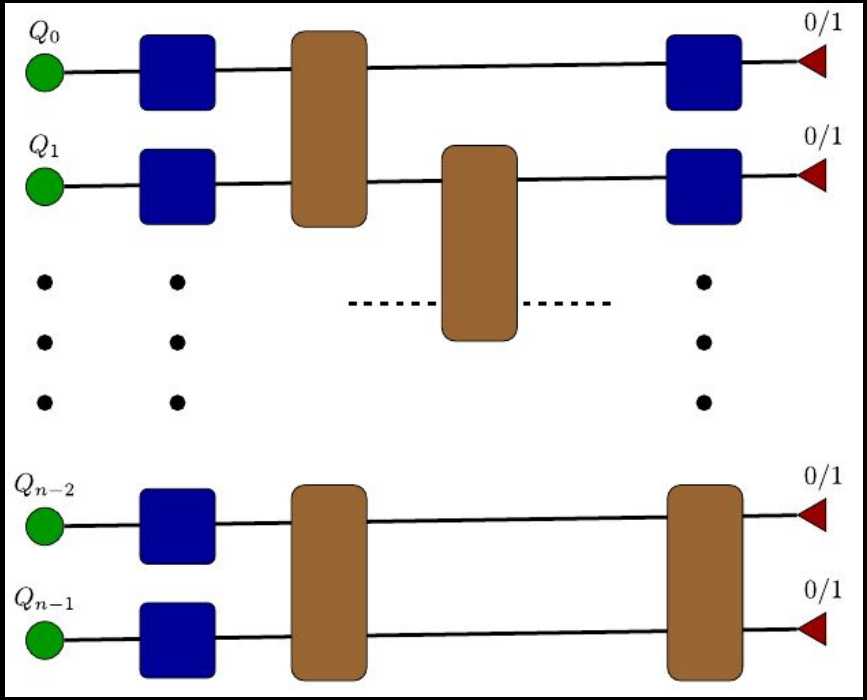
Fonte: Nvidia

Armazenamento de forma eficiente

- Identificar cancelamento($AA^{\dagger} = I$);
- Compactação de tensores dispersos ou diagonais;
- Uso de Aceleradores (GPU);
- Caching;
- Definição de tipos conhecidos de tensors.

O que pode ser extraido com TN?

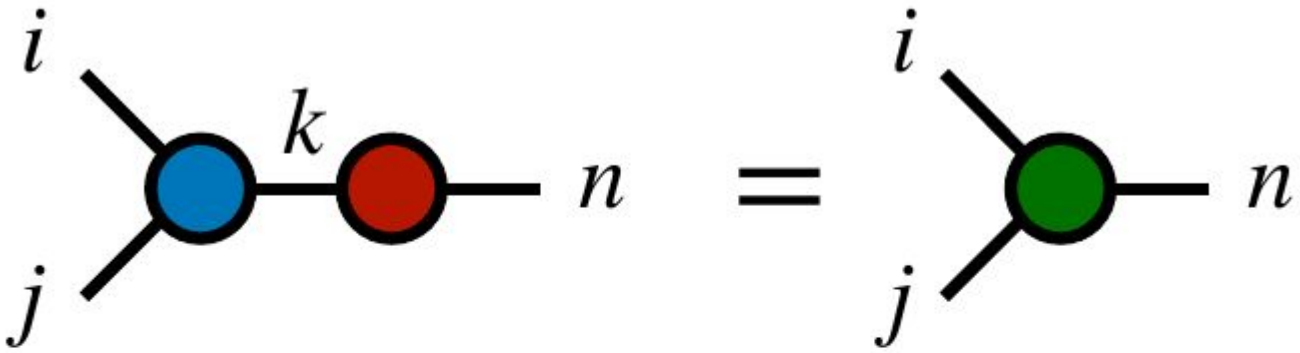
- Density Matrix;
- State Vector;
- Expectation Value;
- Unitary;
- Tudo depende de como é feita a contração.



Fonte: https://doi.org/10.1145/3547334

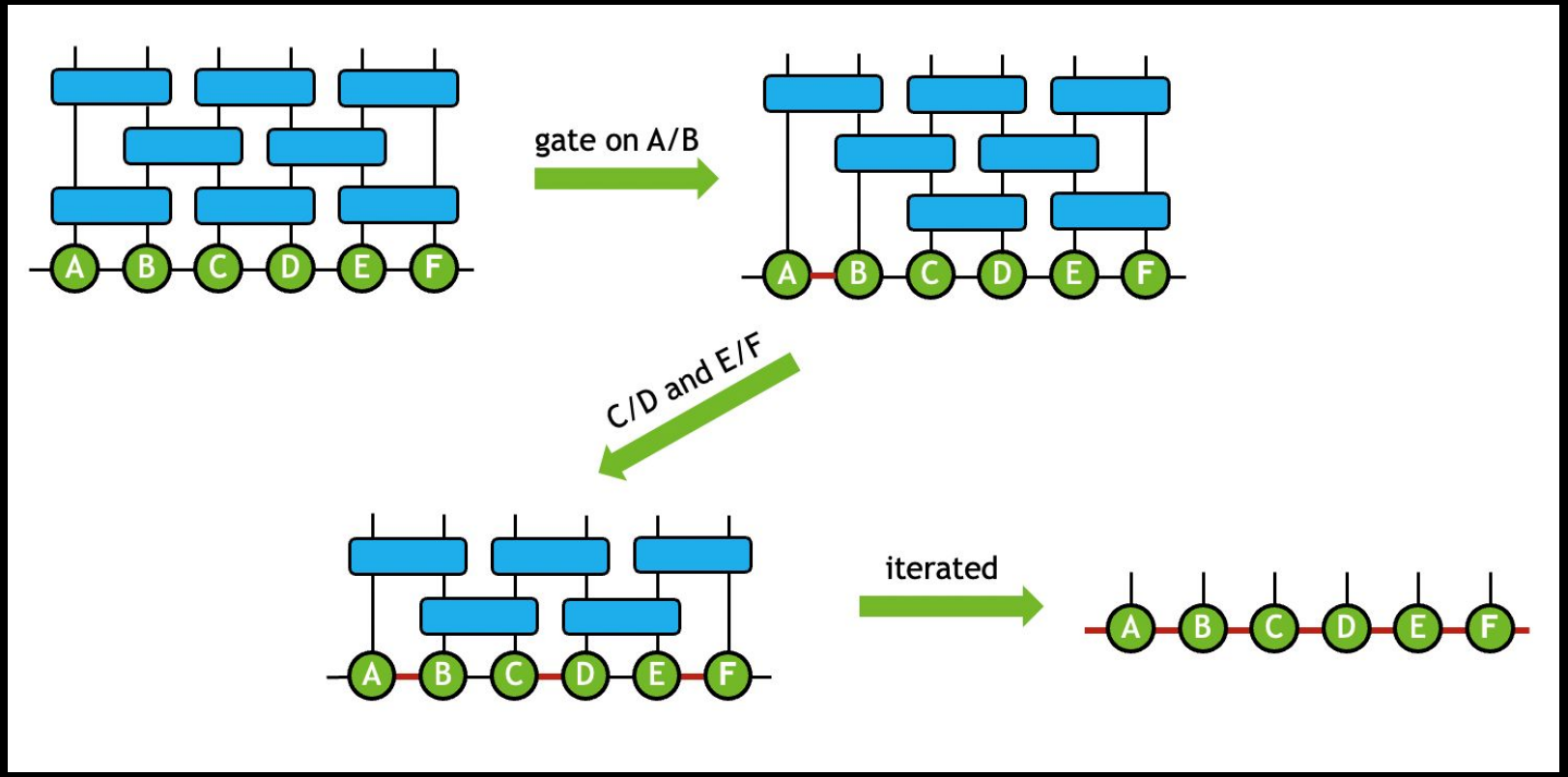
Operações

- Contrações (Unir tensores em blocos);
  - Sequência de contrações garante resultado mais ou menos custoso;
- Decomposição (Inverso da contração);
  - SVD;
  - QR;
- Existem também outras operações como Slicing → Possibilita paralelização.

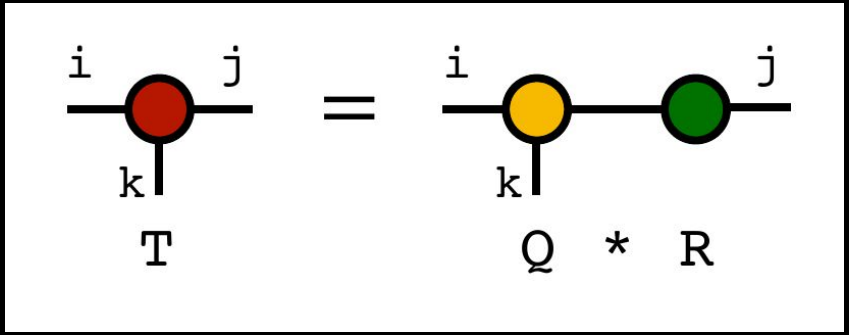


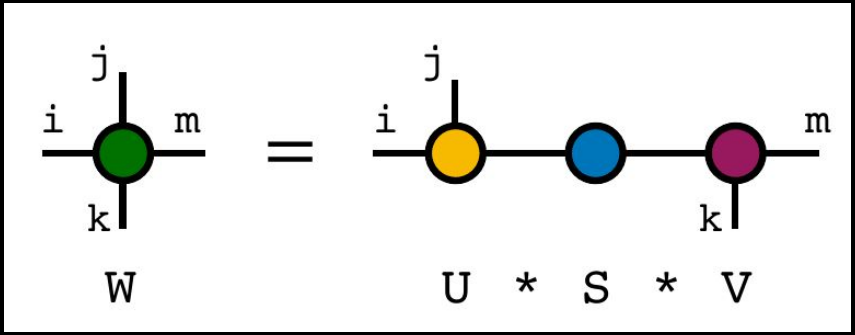
$$\sum_k T_{ijk} M_{kn} = R_{ijn}$$

Fonte: arXiv:2007.14822

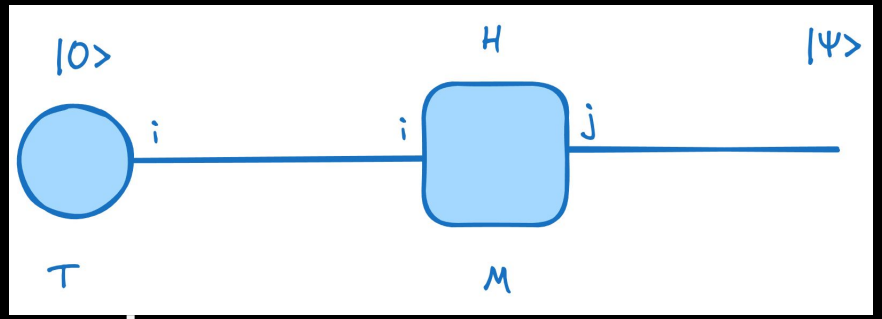


Fonte: Nvidia





Fonte: arXiv:2007.14822



$$T = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad M = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$Q_j = \sum_i T_i M_{ij}$$

$$Q_0 = T_0 M_{00} + T_1 M_{10} = \frac{1}{\sqrt{2}}((1)(1) + (0)(1))$$

$$Q_1 = T_0 M_{01} + T_1 M_{11} = \frac{1}{\sqrt{2}}((1)(1) + (0)(-1))$$

$$Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Fonte: Criação própria via Excalidraw e Overleaf

Custo de operações

- Medido em FLOPs;
- Bond dimensions → principal contribuinte;
  - Máximo *D* que um index pode ser;
  - Representa a quantidade de entanglements;
- Problema de otimização;
- Análise espaço e tempo.

$$Q_0 = T_0 M_{00} + T_1 M_{10} = \frac{1}{\sqrt{2}}((1)(1) + (0)(1))$$

$$Q_1 = T_0 M_{01} + T_1 M_{11} = \frac{1}{\sqrt{2}}((1)(1) + (0)(-1))$$

D = 1; d = 2

$O(d) * O(d) = O(d^2)$

FLOPs = d (2d - 1)

$$\mathrm{cost_{ij}} = \chi(\mathrm{i})\chi(\mathrm{j})\chi(\mathrm{k})\chi(\mathrm{l}) + \chi(\mathrm{j})\chi(\mathrm{k})\chi(\mathrm{l})\chi(\mathrm{m})\chi(\mathrm{n}).$$

$$N_{klmn} = \sum_{ij} T_{ijkl} X_i Y_{jmn}$$

IJ

$$Q_{jkl} = \sum_i T_{ijkl} X_i \rightarrow N_{klmn} = \sum_j Q_{jkl} Y_{jmn}$$

JI

$$Q'_{iklmn} = \sum_j T_{ijkl} Y_{jmn} \rightarrow N_{klmn} = \sum_i Q'_{iklmn} X_i$$

$$\begin{aligned}\mathrm{cost_{ji}} = {} & \chi(i)\chi(j)\chi(k)\chi(l)\chi(m)\chi(n) \\ & + \chi(i)\chi(k)\chi(l)\chi(m)\chi(n).\end{aligned}$$

Fonte: arXiv:2001.08063

Métodos para encontrar a sequência

- Greedy;
- Algoritmos genéticos (métodos heurísticos);
- Simulated annealing, Tabu Search, DFS, BFS (busca em grafos);
- QuickBB;
- Outros métodos que minimizam a função de custo.

Passos para Simular o Circuito

1. Definição da rede;
2. SImplificação da rede;
3. Busca pelo melhor caminho de contração;
4. Aplicação das contrações → resultado.

Obs: Grafos são dispersos (circuitos não são completamente interligados)

Simplificação

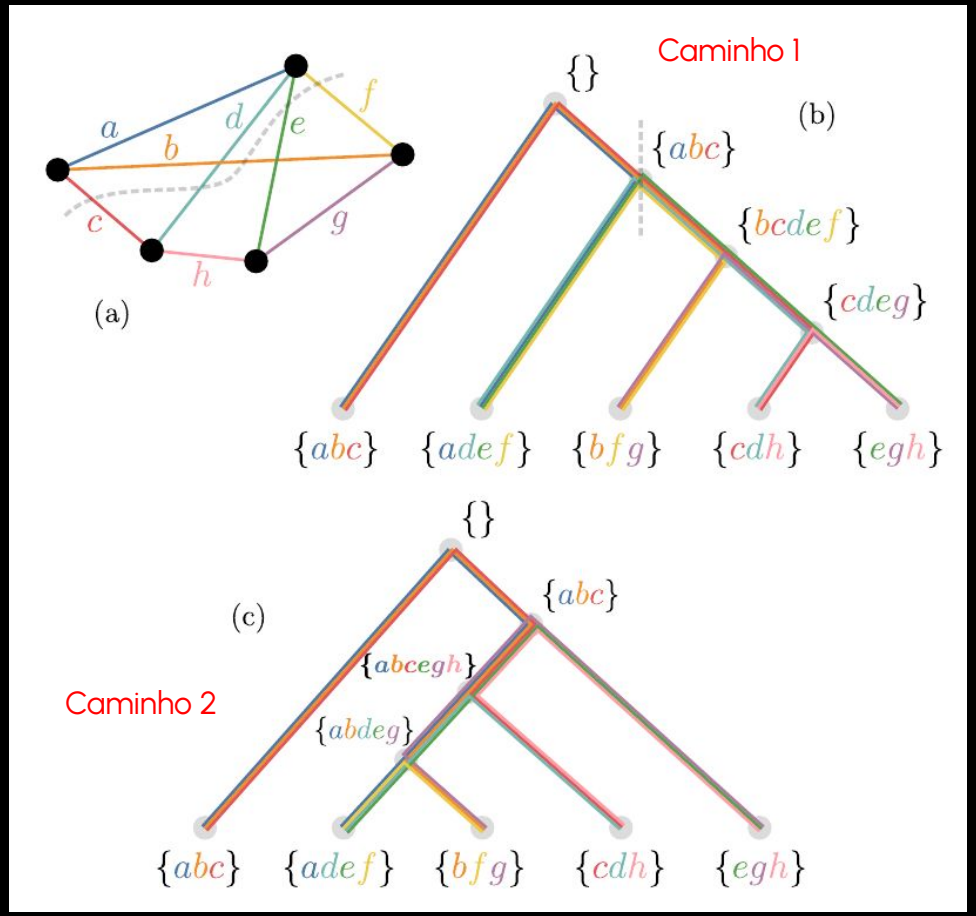
- Depende da estrutura da rede;
- Simplificações → Absorção de tensors, SVD, Introdução de Hyperedges, etc. Tidos como ≈ O(1) (tempo);
- Complexidade tempo:
  - O(d x n) ≈ O(n);
  - número de passos d é geralmente pequeno;
  - nodes n é mais expressivo;
- Espaço:
  - Uso de armazenamento reduz a cada iteração.

while there are simplifications **do**
  for node in graph **do**
    $Simplify_1(node)$
    $Simplify_2(node)$
    $Simplify_3(node)$
    ...
  end for
end while

Fonte: Criação própria via Overleaf

Busca pelo melhor caminho de contração (Hyper-Par)

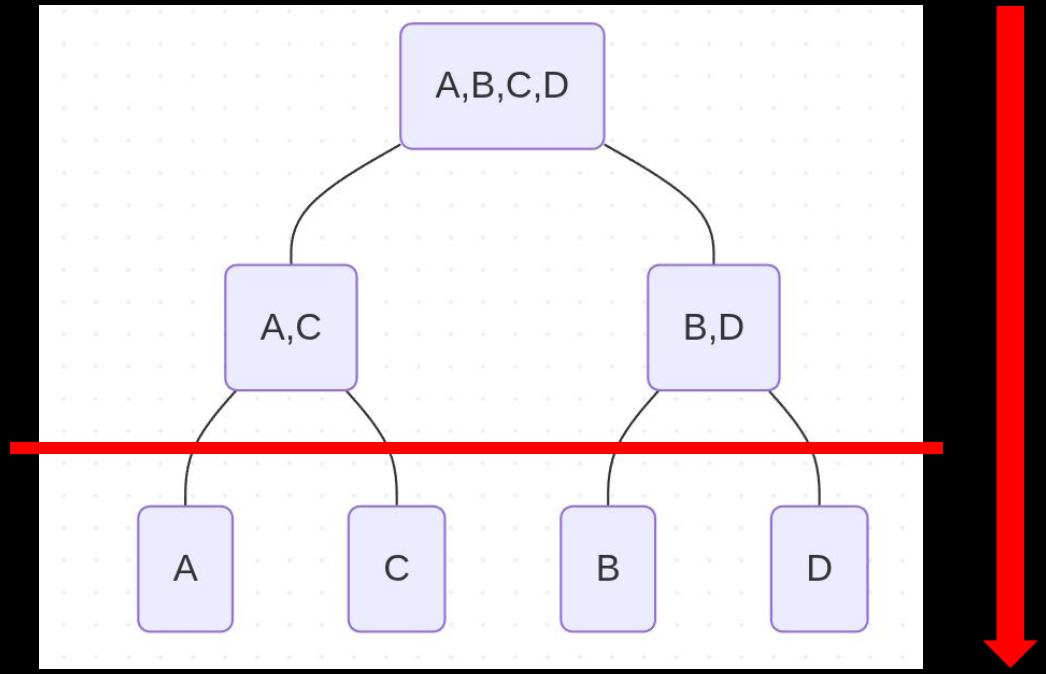
- Repartição de subárvores → KaHyPar;
- Uso de outros algoritmos nas subárvores (Optimal ou Greedy);
- Parâmetros de balanceamento (ε), repartições (k) e cut-off.



Fonte: arXiv:2002.01935v4

Todo o Grafo

Esse é o caminho das contrações. Melhor repartição dará o melhor caminho.



Partições

Fonte: Criação própria via Mermaid

Hyper-Par

- Usar a recursão e ajuste de parâmetros;
  - Pega samples dos resultados e o menor custo “ganha”;
  - Minimização do corte (menos indices);
- Gera problemas tão pequenos que algoritmos exatos são rápidos;
- Utiliza de algoritmos óptimos para criar o restante;
- Resultados próximos ao ótimo relativamente rápido.

Hyper-Par Algoritmo

- Testa combinações de parâmetros;
- Complexidade Tempo:
  - O(k * ε * cut * (V + cost));
    - O(V + cost);
  - Partition é a parte mais custosa;
- Complexidade Espaço:
  - O(n) → (O(G) = O(n));
  - O(n).

```
for each combination (k, ϵ, cut) do
    V ← Partition(G, cut, ϵ, k)
    cost ← Cost(V)
    if costB then
        B ← cost
        BG ← V
    end if
end for
```

Fonte: Criação própria via Overleaf

- Custo cresce exponencialmente com o número de indices;
- Complexidade tempo:
  - O(n) → sendo n a quantidade de nós.
- Complexidade Espaço:
  - O(1) → valor constante retornado.

```
function COST(Node)
    if Node is leaf then
        return 0
    end if

    c ← 2^indexes
    total ← c

    for child in node.children do
        total ← total + Cost(child)
    end for

    return total
end function
```

Fonte: Criação própria via Overleaf

- Criação da árvore via recursão (bipartição);
- Complexidade Tempo:
  - KaHyPar sempre executa;
  - Opt é custoso (quando n grande → não deve acontecer);
  - j * O(KaHyPar) + h* O(Opt)
    - O(KaHyPar);
- Complexidade Espaço:
  - Children é o principal;
  - O(n);

```
function PARTITION(V,cut,ϵ,k)
    if |V| ≤ 1 then
        return Leaf(V)
    end if
    if |V| ≤ cut then
        return Opt(V)
    end if

    parts ← KaHyPar(V, ϵ, k)
    children ← {}

    for each V_i in parts do
        children.append(Partition(V_i, cut, ϵ, k))
    end for
    return Node(V, children)
end function
```

Fonte: Criação própria via Overleaf

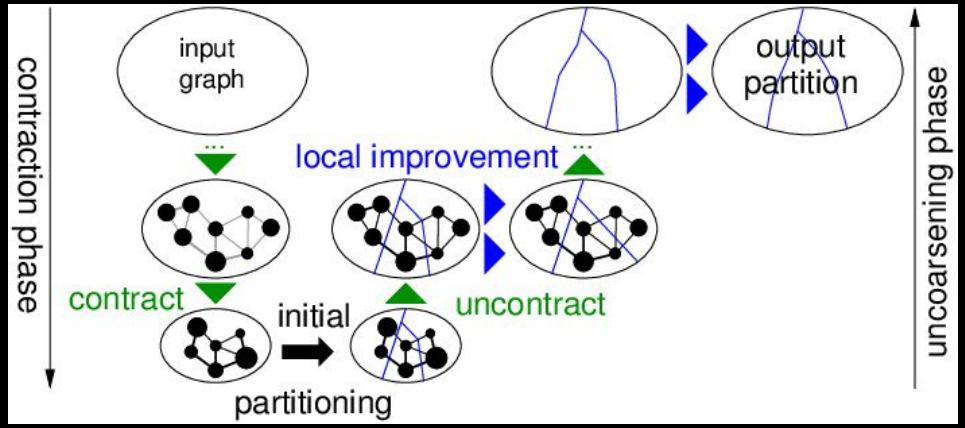
Hyper-Greedy Opt.

- Calcula o custo de cada nó (tamanho do tensor);
- Dá uma probabilidade para ter a chance de escolher um pior → fugir do local optima;
- Complexidade tempo:
  - $O(n^3)$;
- Complexidade Espaço:
  - $O(n^2)$ → pares;
- Por isso n deve ser pequeno.

```
function OPT(V)
    outNodes ← {}
    nodes ← every item in V as Leaf
    while len(nodes) > 1 do
        pairs ← {}
        costs ← {}

        for each pair (i, j) in nodes do
            cost ← size(node_i) − α(size(node_i) + size(node_j))
            pairs.append((i, j))
            costs.append(cost)
        end for

        probs ← {}
        Z ← 0
        for cost in costs do
            p ← exp(−cost/τ)
            probs.append(p)
            Z ← Z + p
        end for

        for k in range(len(probs)) do
            probs_k ← probs_k/Z
        end for
        i, j ← randomChoice(pairs, probs)
        newNode ← Node(nodes_i, nodes_j)
        remove nodes_i, nodes_j from nodes
        outNodes.append(newNode)
    end while
    return outNodes
end function
```

Fonte: Criação própria via Overleaf

- Multilevel graph partitioning (KaHyPar);
  1. Reduz o grafo;
  2. Particiona;
  3. Realiza o caminho inverso;
- Usa múltiplos algoritmos (Portifólio);
- Hypergraphs;
- Pode usar pré-processamento e otimizações.



Fonte: ResearchGate

- Reduz o grafo contraindo os nós;
    - Fase de Coarsening;
- Faz shuffle dos nós para ser mais "justo";
- Procura por pequenos clusters;
- Para com algum critério específico (não há mais nós, antingiru certo tamanho, etc.);

```
nodes ← shuffle(G.nodes)
matched ← {}

for node in nodes do
    if node in matched then
        continue
    end if
    bestNeighbor ← null
    maxRating ← −∞

    neighbors ← neighborsLocalClusters(node)
    for neighbor in neighbors do
        if neighbor == node || neighbor in matched || weight_node +
weight_neighbor > maxVertexWeight then
            continue
        end if

        r ← rate(node, neighbor)
        if r > maxRating then
            maxRating ← r
            bestNeighbor ← neighbor
        end if
    end for

    if bestNeighbor then
        contract(node, bestNeighbor)
        matched.add(node, bestNeighbor)
    end if
end for
```

Fonte: Criação própria via Overleaf

Tempo = $O(e_n v_e)$
Espaço = $O(g)$

Tempo = $O(e_s = e_n + e_g)$
Espaço = $O(e_s)$

Tempo = $O(e_g)$
Espaço → Diminui ($O(1)$)

```
function NEIGHBORSLOCALCLUSTERS(node)
    neighbors ← {}
    for edge in incidentEdges_node do
        if len(edge.vertices) ≤ threshold then

            for neighbor in edge.vertices do
                if neighbor ≠ node then
                    neighbors.add(neighbor)
                end if
            end for

        end if
    end for

    return neighbors
end function

function RATE(node, neighbor)
    s ← incidentEdges_node ∩ incidentEdges_neighbor
    return Σ_e^s 1/len(e.vertices) − 1
end function

function CONTRACT(node, neighbor)
    node.weight ← node.weight + neighbor.weight
    for edge in incidentEdges_neighbor do
        edge.removeVertex(neighbor)
        if ! node in edge.vertices then
            edge.addVertex(node)
            node.addIncidentEdge(edge)
        end if

        if len(edge.vertices) < 2 then
            removeEdge(edge)
        end if
    end for
    removeVertex(neighbor)
end function
```

Fonte: Criação própria via Overleaf

n=nodes; g=neighbors; e=edges; v=vertices;

- $O(n * (e_n v_e + g e_s + e_g))$

Todos os nós conectados (threshold alto):

- e_n=n; v_e=n; g=n-1; e_s=n; e_g=n;
- $O(n^3)$;

Caso mais real:

- Todos os valores são pequenos, dado grafos mais esparsos (circuitos geralmente não são totalmente interligados) , pré-processados e threshold baixo;
- Tempo → O(n);
- Espaço → O(n).

```
nodes ← shuffle(G.nodes)
matched ← {}

for node in nodes do
    if node in matched then
        continue
    end if
    bestNeighbor ← null
    maxRating ← −∞

    neighbors ← neighborsLocalClusters(node)
    for neighbor in neighbors do
        if neighbor == node || neighbor in matched || weight_node +
weight_neighbor > maxVertexWeight then
            continue
        end if

        r ← rate(node, neighbor)
        if r > maxRating then
            maxRating ← r
            bestNeighbor ← neighbor
        end if
    end for

    if bestNeighbor then
        contract(node, bestNeighbor)
        matched.add(node, bestNeighbor)
    end if
end for
```

Fonte: Criação própria via Overleaf

Tempo = O(n log n)
Espaço = O(n)

```
function RECURSIVEBIPARTITION(G, k)
    if k ≤ 1 then:
        return {}
    end if
    V0, V1 ← 2wayPartition(G)
    G0 ← new graph with only V0
    G1 ← new graph with only V1
    left ← recursiveBipartition(G0, k/2)
    right ← recursiveBipartition(G1, k/2)
    return left + right
end function
```

Tempo = O(n log n)
Espaço = O(n)

```
function 2WAYPARTITION(G)
    hierarchy ← {G}
    currentG ← G
    while currentG.numVertices > threshold do
        newG ← coarse(currentG)
        hierarchy.append(newG)
        currentG ← newG
    end while

    cut ← partition(currentG)

    while hierarchy do
        coarsedG ← hierarchy.pop()
        cut ← projectCut(cut, coarsedG)
    end while
    return cut
end function
```

Considerando coarsening ≈ $1/2^k$

Tempo = O(n)
Espaço = O(n)

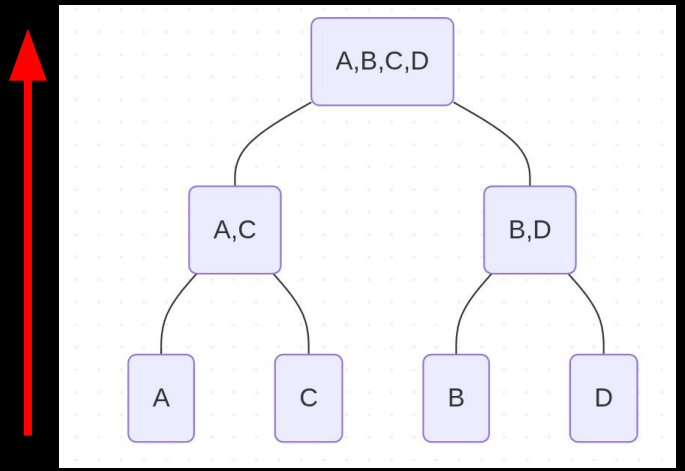
```
function PROJECTCUT(cut, G)
    newPartition ← {}
    for node in G.nodes do
        SU ← node.parent
        newPartition_node ← cut_SU
    end for
    return newPartition
end function
```

Tempo = O(n log n)
Espaço = O(n)

```
function PARTITION(G)
    bestSize ← ∞
    bestPartition ← null

    for attempt in range(attempts) do
        rand0, rand1 ← Two random nodes from G
        block0 ← {rand0}
        block1 ← {rand1}

        candidates0 ← PriorityQueue(rand0.neighbors)
        candidates1 ← PriorityQueue(rand1.neighbors)
        targetWeight ← G.totalWeight/2

        while len(block0) + len(block1) < G.vertices do
            currentBlock ← Current Smaller block
            currentQueue ← Queue relative to the current block

            if ! currentQueue.empty then
                maxNode ← currentQueue.popMaxGainNode()
                if ! maxNode in block0 && ! maxNode in block1 then
                    currentBlock.add(maxNode)

                    for neighbor in maxNode.neighbors do
                        if ! neighbor in block0 && ! neighbor in block1 then
                            currentQueue.insert(neighbor)
                        end if
                    end for

                end if
            else
                currentBlock.add(G.getRandomUnassignedNode())
            end if
        end while

        partition ← (block0, block1)
        if partition.cutSize < bestSize then
            bestSize ← partition.cutSize
            bestPartition ← partition
        end if
    end for
    return bestPartition
end function
```

Fonte: Criação própria via Overleaf

- A contração segue a árvore criada;
- Tempo → $O(n * 2^D)$ sendo n o número de nós e D a dimensão de ligação dos tensors;
- Espaço → $O(2^D)$ para o armazenamento dos tensors.




```
function CONTRACTNET(node)
    if node is a Leaf then
        return contractLocal(node.tensors)
    end if
    leftTensor ← contractNet(node.left)
    rightTensor ← contractNet(node.right)
    return contract(leftTensor, rightTensor)
end function
```

Fonte: Criação própria via Overleaf e Mermaid

Considerações Finais

- Para encontrar o melhor caminho o limitante é o KaHyPar ≈ O(n log n) do qual é repetido múltiplas vezes;
- Hyper-Par é limitado pelo tamanho dos tensors durante a contração;
- Quando D é pequeno o tensor é menos disperso então KaHyPar é o maior peso.
- Em espaço o algoritmo consome no pior caso $O(2^D)$. Quando D é pequeno O(n);
- Uso de Paralelização e aceleradores;
- Uso de múltiplas funções de otimização.

Repositório

https://github.com/Dpbm/mestrado-aulas/tree/main/algoritmos/projeto-seminario

Referências

FISHMAN, M.; WHITE, S.; STOUDENMIRE, E. The ITensor Software Library for Tensor Network Calculations. SciPost Physics Codebases, p. 004, 23 ago. 2022.

Overview — NVIDIA cuQuantum 23.06.0 documentation. Disponível em: <https://docs.nvidia.com/cuda/cuquantum/23.06.0/cutensornet/overview.html>. Acesso em: 4 abr. 2026.

SCHINDLER, F.; JERMYN, A. S. Algorithms for tensor network contraction ordering. Machine Learning: Science and Technology, v. 1, n. 3, p. 035001, 2 jul. 2020.

BRIDGEMAN, J. C.; CHUBB, C. T. Hand-waving and interpretive dance: an introductory course on tensor networks. v. 50, n. 22, p. 223001–223001, 9 mar. 2016.

NGUYEN, T. et al. Tensor Network Quantum Virtual Machine for Simulating Quantum Circuits at Exascale. ACM Transactions on Quantum Computing, v. 4, n. 1, p. 1–21, 21 out. 2022.

ORÚS, R. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. Annals of Physics, v. 349, p. 117–158, out. 2014.

GRAY, J.; KOURTIS, S. Hyper-optimized tensor network contraction. Quantum, v. 5, p. 410, 15 mar. 2021.

SCHLAG, S. et al. High-Quality Hypergraph Partitioning. ACM Journal of Experimental Algorithmics, v. 27, p. 1–39, 21 abr. 2022.